

BGlabs3

姓名	杨力泉
学号	231220069
日期	2026 年 5 月 4 日

一、本地实验环境

- 操作系统：Ubuntu 22.04 (WSL2)
- JDK 版本：OpenJDK 1.8.0_281
- Hadoop 版本：3.2.1
- HBase 版本：2.4.0
- 硬件配置：vCPU2核，内存4GB，硬盘30G
- 平台环境：集群环境（实验平台）

二、任务一：基于 HBase 的用户画像管理

2.1 设计思路

- 本任务要求使用 HBase Shell 创建用户画像表并进行 CRUD 操作。核心设计如下：
 - 表名：`user_profile`
 - RowKey：`user_id` (用户ID)
 - 列族：`base_info` (基础信息) 和 `social_stats` (社交数据)，将静态属性与动态属性分离。
 - 利用 HBase 的稀疏存储特性，不同用户可拥有不同的列；利用动态列扩展特性，为特定用户直接新增 `phone` 字段而不需修改表结构。

2.2 操作步骤与结果截图

① 建表 创建包含 base_info 和 social_stats 两个列族的表。

```
hk416@hk416-virtual-machine:~$ hbase shell
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
HBase Shell
Use "help" to get list of supported commands.
Use "exit" to quit this interactive shell.
For Reference, please visit: http://hbase.apache.org/2.0/book.html#shell
Version 2.4.0, r282ab70012ae843af54a6779543ff20acbcbb629, Thu Dec 3 09:58:52 PST 2020
Took 0.0010 seconds
hbase:001:0> create 'user_profile', 'base_info', 'social_stats'
Created table user_profile
Took 2.9193 seconds
=> Hbase::Table - user_profile
```

② 插入数据 向 u001、u002、u003 插入测试数据，其中 u003 无粉丝数记录。

```

hbase:002:0> put 'user_profile', 'u001', 'base_info:name', 'ZhangSan'
'social_stats:followers', '850'
put 'user_profile', 'u003', 'base_info:name', 'WangWu'
put 'user_profile', 'u003', 'base_info:age', '30'
put 'user_profile', 'u003', 'base_info:sex', 'female' Took 0.0954 seconds

hbase:003:0> put 'user_profile', 'u001', 'base_info:age', '25'
Took 0.0288 seconds
hbase:004:0> put 'user_profile', 'u001', 'base_info:sex', 'male'
Took 0.0223 seconds
hbase:005:0> put 'user_profile', 'u001', 'social_stats:followers', '1200'
Took 0.0144 seconds
hbase:006:0> put 'user_profile', 'u002', 'base_info:name', 'LiSi'
Took 0.0084 seconds
hbase:007:0> put 'user_profile', 'u002', 'social_stats:followers', '850'
Took 0.0101 seconds
hbase:008:0> put 'user_profile', 'u003', 'base_info:name', 'WangWu'
Took 0.0069 seconds
hbase:009:0> put 'user_profile', 'u003', 'base_info:age', '30'
Took 0.0091 seconds
hbase:010:0> put 'user_profile', 'u003', 'base_info:sex', 'female'
Took 0.0187 seconds

```

数据插入后的结果展示：

```

hbase:011:0> scan 'user_profile'
ROW                                COLUMN+CELL
u001                               column=base_info:age, timestamp=2026-04-28T19:32:00.543, value=25
u001                               column=base_info:name, timestamp=2026-04-28T19:32:00.450, value=ZhangSan
u001                               column=base_info:sex, timestamp=2026-04-28T19:32:00.621, value=male
u001                               column=social_stats:followers, timestamp=2026-04-28T19:32:00.710, value=1200
u002                               column=base_info:name, timestamp=2026-04-28T19:32:00.779, value=LiSi
u002                               column=social_stats:followers, timestamp=2026-04-28T19:32:00.827, value=850
u003                               column=base_info:age, timestamp=2026-04-28T19:32:00.908, value=30
u003                               column=base_info:name, timestamp=2026-04-28T19:32:00.867, value=WangWu
u003                               column=base_info:sex, timestamp=2026-04-28T19:32:05.734, value=female
3 row(s)
Took 0.1093 seconds

```

③ **粉丝数查询** 扫描有 social_stats 列族数据的用户，体现了稀疏存储：u101 无此列族信息则不返回。

```

hbase:012:0> scan 'user_profile', {COLUMNS => 'social_stats'}
ROW                                COLUMN+CELL
u001                               column=social_stats:followers, timestamp=2026-04-28T19:32:00.710, value=1200
u002                               column=social_stats:followers, timestamp=2026-04-28T19:32:00.827, value=850
2 row(s)
Took 0.0393 seconds

```

④ **动态新增字段及精确查询** 为 u001 用户动态新增 base_info:phone 字段，并通过 Get 命令精确查询。

```

hbase:013:0> put 'user_profile', 'u001', 'base_info:phone', '13800138000'
Took 0.0078 seconds
hbase:014:0> get 'user_profile', 'u001', 'base_info:phone'
COLUMN                            CELL
base_info:phone                    timestamp=2026-04-28T19:40:08.399, value=13800138000
1 row(s)
Took 0.0345 seconds

```

```
hbase:016:0> scan 'user_profile', {COLUMNS => 'base_info:phone'}
ROW
u001                                COLUMN+CELL
                                column=base_info:phone, timestamp=2026-04-28T19:40:08.399, value=13800138000
1 row(s)
Took 0.0516 seconds
```

⑤ **删除表** 实验结束后禁用并删除 user_profile 表。

```
hbase:017:0> disable 'user_profile'
Took 2.3260 seconds
hbase:018:0> drop 'user_profile'
Took 0.3701 seconds
hbase:019:0> list
TABLE
0 row(s)
Took 0.0572 seconds
=> []
```

三、任务二：基于 MapReduce 的播放时长统计

3.1 设计思路与核心代码

- **核心目标**：统计各个视频分区下，所有播放行为（action_type=1）的总播放时长。
- **Mapper 设计**：读取日志行，按空格切分，过滤 action_type='1' 的记录，输出 `<video_category, play_duration>`。

```
public static class PlayDurationMapper
    extends Mapper<LongWritable, Text, Text, LongWritable> {

    private Text category = new Text();
    private LongWritable duration = new LongWritable();

    @Override
    protected void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {
        String line = value.toString();
        String[] fields = line.split(" ");
        if (fields.length >= 10 && "1".equals(fields[4])) {
            try {
                String cat = fields[5];           // video_category
                long dur = Long.parseLong(fields[1]); // play_duration
                category.set(cat);
                duration.set(dur);
                context.write(category, duration); // (种类, 时长)
            } catch (NumberFormatException e) {
            }
        }
    }
}
```

- **Reducer 设计**：对同一视频分区的所有 play_duration 累加求和，输出 <video_category, total_duration>。

```
public static class SumReducer
    extends Reducer<Text, LongWritable, Text, LongWritable> {

    private LongWritable result = new LongWritable();

    @Override
    protected void reduce(Text key, Iterable<LongWritable> values, Context context)
        throws IOException, InterruptedException {
        long sum = 0;
        for (LongWritable val : values) {
            sum += val.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}
```

3.2 MapReduce 提交

提交：

1

231220069a

yarn jar /home/231220069a/playduration...

MR

2026/5/4 02:21:05

2026/5/4 02:21:27

成功

查看日志

2026-05-03 14:21:07,682 INFO client.RMProxy: Connecting to ResourceManager at master02/192.168.100.200:8032

2026-05-03 14:21:08,519 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your a pplication with ToolRunner to remedy this.

2026-05-03 14:21:08,533 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/231220069a/.staging/job_1774860907544_1895

2026-05-03 14:21:08,638 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false

2026-05-03 14:21:08,887 INFO input.FileInputFormat: Total input files to process : 1

2026-05-03 14:21:08,953 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false

2026-05-03 14:21:08,996 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false

2026-05-03 14:21:09,005 INFO mapreduce.JobSubmitter: number of splits:1

2026-05-03 14:21:09,218 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false

2026-05-03 14:21:09,251 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1774860907544_1895

2026-05-03 14:21:09,251 INFO mapreduce.JobSubmitter: Executing with tokens: []

2026-05-03 14:21:09,601 INFO conf.Configuration: found resource resource-types.xml at file:/home/workspace/hadoop-3.2.1/etc/hadoop/resource-types.xml

2026-05-03 14:21:09,624 INFO resource.ResourceUtils: Adding resource type - name = yarn.io/gpu, units = , type = COUNTABLE

2026-05-03 14:21:09,732 INFO impl.YarnClientImpl: Submitted application application_1774860907544_1895

2026-05-03 14:21:09,789 INFO mapreduce.Job: The url to track the job: http://114.212.130.202:8088/proxy/application_1774860907544_1895/

2026-05-03 14:21:09,790 INFO mapreduce.Job: Running job: job_1774860907544_1895

2026-05-03 14:21:15,879 INFO mapreduce.Job: Job job_1774860907544_1895 running in uber mode : false

2026-05-03 14:21:15,881 INFO mapreduce.Job: map 0% reduce 0%

2026-05-03 14:21:20,958 INFO mapreduce.Job: map 100% reduce 0%

2026-05-03 14:21:25,990 INFO mapreduce.Job: map 100% reduce 100%

YARN Web 监控界面：

application_1774860907544_1895	231220069a	Play Duration Sum by Category	MAPREDUCE	2025class02	0	Mon May 4 02:21:09 +0800 2026	Mon May 4 02:21:09 +0800 2026	Mon May 4 02:21:24 +0800 2026	FINISHED	SUCCEEDED	N/A
--------------------------------	------------	-------------------------------	-----------	-------------	---	-------------------------------	-------------------------------	-------------------------------	----------	-----------	-----

3.3 Hive 外部表挂载验收与执行结果

为了验证 MapReduce 输出结果，创建了 Hive 外部表指向 HDFS 输出目录：

```
CREATE EXTERNAL TABLE IF NOT EXISTS play_duration_summary(
    category STRING COMMENT '视频分区',
    total_duration BIGINT COMMENT '总播放时长'
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY '\t'
STORED AS TEXTFILE
LOCATION '/Exp3/output';
```

通过

SQL 查询验证统计结果是否正确：

```
1
2 SELECT * FROM play_duration_summary;
3
```

当前执行结果 历史执行语句 已保存的语句

Play Duration Summary.category	Play Duration Summary.total Duration
dance	2998864
education	3056394
gaming	3023813
music	3019455
news	3026095
sports	3010164
tech	2954295

四、任务三：基于 Hive 的数仓分层与 HQL 数据分析

4.1 设计思路

- **数据源映射**：创建外部表 `behavior_log_raw` 挂载 HDFS 日志，确保不移动原始文件。
- **分区表优化**：创建 `log_part` 表，以 `video_category` 为分区键，使后续按分区的查询裁剪数据。
- **分桶表优化**：创建 `log_bucket` 表，按 `video_id` 分 3 个桶，为后续可能的表连接 (JOIN) 或抽样 (SAMPLE) 提供更高效率的物理存储。
- **业务查询**：基于 `log_part` 表执行三类典型需求：分区过滤查询、用户设备风控统计、分区点赞聚合。

4.2 操作步骤与结果截图

① 创建原始数据外部表

3253	hive	231220069a	CREATE EXTERNAL TABLE IF NOT EXISTS behavior_log_raw(video_id STRING, play_duration BIGINT, video_size BIGINT, action_time STRING, action_type INT, video_category STRING, user_id STRING, device_id STRING, action_count INT, city STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY ' ' STORED AS TEXTFILE LOCATION '/user/231220069a/Exp3/log';	✓	2026/5/4 03:14:31
------	------	------------	--	---	-------------------

② 创建视频分区分区表

3269	hive	231220069a	CREATE TABLE IF NOT EXISTS log_part(video_id STRING, play_duration BIGINT, video_size BIGINT, action_time STRING, action_type INT, user_id STRING, device_id STRING, action_count INT, city STRING) PARTITIONED BY (video_category STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY ' ' STORED AS TEXTFILE;	✓	2026/5/4 03:32:42
3270	hive	231220069a	INSERT OVERWRITE TABLE log_part PARTITION(video_category) SELECT video_id, play_duration, video_size, action_time, action_type, user_id, device_id, action_count, city, video_category FROM behavior_log_raw;	✓	2026/5/4 03:33:39

③ 创建视频ID分桶表

3271	hive	231220069a	CREATE TABLE IF NOT EXISTS log_bucket(video_id STRING, play_duration BIGINT, video_size BIGINT, action_time STRING, action_type INT, video_category STRING, user_id STRING, device_id STRING, action_count INT, city STRING) CLUSTERED BY (video_id) INTO 3 BUCKETS ROW FORMAT DELIMITED FIELDS TERMINATED BY ' ' STORED AS TEXTFILE;	✓	2026/5/4 03:34:57
3272	hive	231220069a	INSERT OVERWRITE TABLE log_bucket SELECT * FROM behavior_log_raw;	✓	2026/5/4 03:36:22

④ 业务查询 A：查询科技区（tech）数据

1	SELECT * FROM log_part WHERE video_category = 'tech' LIMIT 100;
---	---

当前执行结果	历史执行语句	已保存的语句							
Log Part.video Id	Log Part.play Duration	Log Part.video Size	Log Part.action Time	Log Part.action Type	Log Part.user Id	Log Part.device Id	Log Part.action Count	Log Part.city	Log Part.video Category
v0278	573	94	2026-04-10T08:00:14	1	u3866	d3866	1	Wuhan	tech
v0440	281	97	2026-04-10T08:00:24	1	u3886	d3886	1	Chengdu	tech
v0017	498	94	2026-04-10T08:00:28	1	u1324	d1324	1	Beijing	tech
v0586	309	45	2026-04-10T08:00:40	2	u0210	d0210	1	Hangzhou	tech
v0438	94	93	2026-04-10T08:00:58	3	u1277	d1277	1	Chengdu	tech
v0393	8	32	2026-04-10T08:01:12	3	u1247	d1247	1	Shenzhen	tech
v0857	273	42	2026-04-10T08:01:24	1	u1147	d1147	1	Shenzhen	tech
v0055	21	15	2026-04-	1	u3717	d3717	1	Shanghai	tech

⑤ 业务查询 B：统计每个用户的不同设备数

1 SELECT user_id, COUNT(DISTINCT device_id) AS device_count

2 FROM log_part

3 GROUP BY user_id

4 LIMIT 100;

当前执行结果

历史执行语句

已保存的语句

User Id	Device Count
u0001	3
u0002	3
u0003	5
u0004	4
u0005	2
u0006	2
u0007	3
u0008	2
u0009	3
u0010	3
u0011	3
u0012	3

⑥ 业务查询 C：统计各分区点赞总数

1 SELECT video_category, SUM(action_count) AS total_likes

2 FROM log_part

3 WHERE action_type = 2

4 GROUP BY video_category

5

当前执行结果

历史执行语句

已保存的语句

Video Category	Total Likes
dance	2848
education	2792
gaming	2764
music	2774
news	2894
sports	2792
tech	2853

五、附件说明

压缩包 231220069_杨力泉_BGlab3.zip 包含：

- 源代码：PlayDurationSum.java
- JAR 包：playduration.jar
- 实验报告：本文件
- 任务一结果文件